ADDITIONAL NOTE: While the third order model from problem 3 did not show overfitting, applying a fourth order model did. Test error and a plot of the curve is shown in problem 3

Problem 1)

1-1)
This is a sum:
$C^T x = c_1 x_1 + c_2 x_2 + \ldots c_n x_n$
Assuming that each state variable is linearly independent:
$\partial/\partial x_i \, C^T x = \partial/\partial x_i \, (c_1 x_1 + c_2 x_2 + \ldots c_n x_n) = 0 + 0 + \ldots c_i \ldots + 0 = c_i$

1-2)
$x^T Q x = \sum_{i=1}^{n} \sum_{j=1}^{n} x_i Q_{ij} x_j$
For the case of x_i, only some of the terms in the matrix contain x_i. Within the sum, there is the case of $i = j$, as well as the case of $i \neq j$.
For $i = j$:
$x_i^T Q x_j = x_i Q_{ij} x_j = x_i Q_{ii} x_i$
$\partial/\partial x_i \, x_i Q_{ii} x_i = 2 x_i Q_{ii}$
When putting this into vector form, this yields:

For $i \neq j$:
$x_i^T Q x_j = \sum_{j=1}^{n} x_i Q_{ij} x_j + \sum_{j=1}^{n} x_j Q_{ji} x_i = (x_i Q_{i1} x_1 + x_i Q_{i2} x_2 \ldots x_i Q_{in} x_n) + (x_1 Q_{1i} x_i + x_2 Q_{2i} x_i \ldots x_n Q_{ni} x_i)$
The term where $i = j$ in these sums is skipped, as that was already covered above. All of these terms only contain x_i to the first power, so when taking the partial derivative, this yields:

$\partial/\partial x_i \sum_{j=1}^{n} x_i Q_{ij} x_j + \sum_{j=1}^{n} x_j Q_{ji} x_i = \sum_{j=1}^{n} Q_{ij} x_j + \sum_{j=1}^{n} x_j Q_{ji} = (Q_{i1} x_1 + Q_{i2} x_2 \ldots Q_{in} x_n) + (x_1 Q_{1i} + x_2 Q_{2i} \ldots x_n Q_{ni})$
Since Q is symmetric:
$(Q_{i1} x_1 + Q_{i2} x_2 \ldots Q_{in} x_n) + (x_1 Q_{1i} + x_2 Q_{2i} \ldots x_n Q_{ni}) = 2(Q_{i1} x_1 + Q_{i2} x_2 \ldots Q_{in} x_n)$
Combining with the case where $i = j$ completes the full vector, as it was previously missing that term. They take the same form, so they can be written in the exact same form after being combined. Similarly, the constant vector from part 1 is also combined yielding:
$\partial/\partial x_i \, x^T Q x = 2(Q_{i1} x_1 + Q_{i2} x_2 \ldots Q_{in} x_n) + c_i = 2Q(i,:)X + c_i$
When taking the derivative with respect to every x_i and concatenating, this yields:
$\nabla x^T Q x = (2Q(1,:)X + c_1) + (2Q(2,:)X + c_2) \ldots (2Q(n,:)X + c_n) = 2QX + C$

Problem 2)
For a convex function, the hessian is positive semidefinite, which for scalar functions translates to a positive second derivative with no discontinuities

2-1)

$f(x) = ax + b \rightarrow d/dx\, f(x) = a \rightarrow d^2/dx^2\, f(x) = 0$
The second derivative is 0 at any value of x, thus it **is convex**, but not strictly convex

2-2)
The power rule for derivatives applies as long as x is positive and a is real
$f(x) = x^a \rightarrow d/dx\, f(x) = ax^{a-1} \rightarrow d^2/dx^2\, f(x) = a(a-1)\, x^{a-2}$
$a \geq 1$ is given in the problem, so:
$a(a-1) \geq 0$
Additionally:
$x^{a-2} \geq 0$ for any x>0 and a, thus:
$d^2/dx^2\, f(x) > 0$
Therefore, the second derivative is 0 or greater and always defined, so the function **is convex**.

2-3)
$f(x) = x\, log(x)$
$d/dx\, f(x) = log(x) + x * 1/x = log(x) + 1$
$d^2/dx^2\, f(x) = 1/x$
Since 1/x > 0 for all x>0, the function **is convex** in x>0

2-4)

$f(x) = ||x||_2 = \sqrt{\sum_{i=1}^{n} x_i^2}$
To prove convexity:
$f(tx_1 + (1-t)x_2) \leq tf(x_1) + (1-t)f(x_2)$
Where 0<t<1. Since the Euclidean norm is a norm, it satisfies the triangle inequality and homogeneity:
$f(u + v) \leq f(u) + f(v)$
$f(cv) = |c|f(v)$
Rearranging the equation by moving t and (1-t) inside the function. This can be done, since both are already positive:
$f(tx_1 + (1-t)x_2) \leq f(tx_1) + f((1-t)x_2)$
This is the form of the triangle inequality:
$u = tx_1, v = (1-t)x_2$
Substituting:

$f(u + v) \leq f(u) + f(v)$
This statement is already known to be satisfied by the triangle inequality, so the Euclidean norm is convex.

Problem 3)
3-1)
$C_i = \theta_0 + \theta_1 \alpha_i$
This is concatenated to yield:
$C = [C_1, C_2 \ldots]^T = [1, \alpha_1; 1, \alpha_2; \ldots] * [\theta_0, \theta_1] = P * \Theta$
Which can be solved by taking the pseudo inverse:
$P^{-1} C = \Theta$
This was solved through code in python and the code is shown at the end of the document. The model was:
$\theta_0 = 6.029e - 03,\ \theta_1 = 3.071e - 04$
The training cost was calculated as the sum of squared error:
$J = 1.471e - 06$

3-2)
$C_D = \theta_0 + \theta_1 \alpha + \theta_2 \alpha^2$
This is concatenated to yield:
$C = [C_1, C_2 \ldots]^T = [1, \alpha_1, \alpha_1^2; 1, \alpha_2, \alpha_2^2; \ldots] * [\theta_0, \theta_1, \theta_3] = P * \Theta$
This can be solved in the same way, and the code is shown at the end.
$P^{-1} C = \Theta$
The model was:
$\theta_0 = 5.904e - 03,\ \theta_1 = 2.589e - 05,\ \theta_3 = 4.688e - 05$
The training cost was:
$J = 1.589e - 07$

3-3)
$C_D = \theta_0 + \theta_1 \alpha + \theta_2 \alpha^2 + \theta_3 \alpha^3$
his is concatenated to yield:
$C = [C_1, C_2 \ldots]^T = [1, \alpha_1, \alpha_1^2, \alpha_1^3; 1, \alpha_2, \alpha_2^2, \alpha_2^3; \ldots] * [\theta_0, \theta_1, \theta_3, \theta_4] = P * \Theta$
This can be solved in the same way, and the code is shown at the end.

$P^{-1}C = \Theta$

The model was:

$\theta_0 = 5.962e-03,\ \theta_1 = 3.770e-05,\ \theta_2 = 3.125e-05,\ \theta_3 = 1.736e-06$

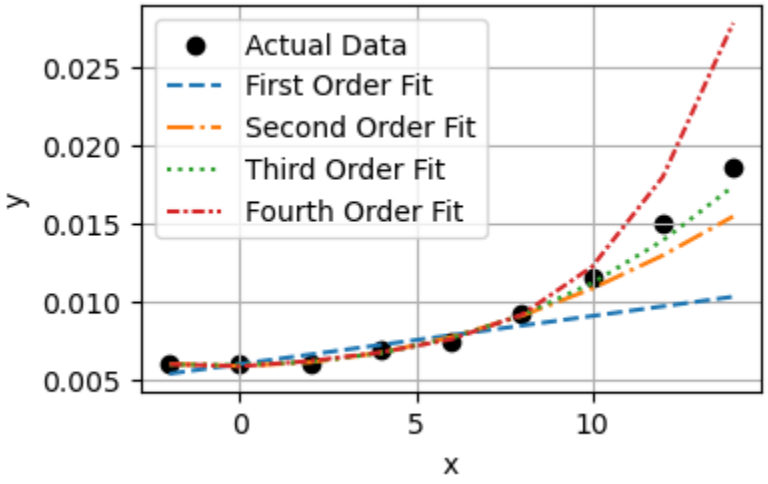
The training cost was:

$J = 1.464e-07$

3-4)

The cost was recalculated as the sum of squared error on only the last three data points and the results are shown below. Training error is the error achieved on the 6 data points used in the least squares fit, while the remaining 3 were used for the test error. A plot of the data points and function fits are shown below that

Model Order	Training Error	Test Error
First	1.471e-06	1.021e-04
Second	1.589e-07	1.447e-05
Third	1.464e-07	2.761e-06
Fourth	1.289e-07	9.406e-05



3-5)

The training error always decreases as the order of the function increases, because more parameters to fit allows the model to fit the data better. The issue is with the test error, as data points which were not included in the training data may not be generalized too. The additional parameters allow for training data to fit better, but by allowing for more parameters, we are assuming something about the function form, which may not be end up being correct in reality. There comes a point where the additional parameters actually cause the fit to get worse for the test data, thus losing generalizability. I would choose the second order model here. The data appears to be parabolic, with no evidence of a cubic relationship, so the third order model should not get picked.

Code:

```
import numpy as np

import pandas as pd
import matplotlib.pyplot as plt
from pandas.plotting import table



x=np.array([-2,0,2,4,6,8,10,12,14])
y=np.array([0.006,0.006,0.006,0.007,0.0075,0.0092,0.0115,0.015,0.0186])
x_train = x[:6]
y_train = y[:6]
x_test = x[6:]
y_test = y[6:]

def to_sci(x):
   return f"{x:.3e}"


#Part 1
P = np.array([x_train**0,x_train**1]).T

P_inv = np.linalg.pinv(P)

# Least squares solution
theta = P_inv @ y_train

# Calculate least squares cost on training data
y_train_pred = P @ theta
cost_train = np.sum((y_train - y_train_pred)**2)
print("First order training cost:", cost_train)
first_order_train = cost_train

# Calculate least squares cost on test data
P_test = np.array([x_test**0, x_test**1]).T
y_test_pred = P_test @ theta
cost_test = np.sum((y_test - y_test_pred)**2)
print("First order test cost:", cost_test)
first_order_test = cost_test
```

```
#Part 2
P = np.array([x_train**0,x_train**1,x_train**2]).T
P_inv = np.linalg.pinv(P)
# Least squares solution
theta = P_inv @ y_train

# Calculate least squares cost on training data
y_train_pred = P @ theta
cost_train = np.sum((y_train - y_train_pred)**2)
print("Second order training cost:", cost_train)
second_order_train = cost_train

# Calculate least squares cost on test data
P_test = np.array([x_test**0, x_test**1, x_test**2]).T
y_test_pred = P_test @ theta
cost_test = np.sum((y_test - y_test_pred)**2)
print("Second order test cost:", cost_test)
second_order_test = cost_test


#Part 3
P = np.array([x_train**0,x_train**1,x_train**2,x_train**3]).T
P_inv = np.linalg.pinv(P)
# Least squares solution
theta = P_inv @ y_train

# Calculate least squares cost on training data
y_train_pred = P @ theta
cost_train = np.sum((y_train - y_train_pred)**2)
print("Third order training cost:", cost_train)
third_order_train = cost_train

# Calculate least squares cost on test data
P_test = np.array([x_test**0, x_test**1, x_test**2, x_test**3]).T
y_test_pred = P_test @ theta
cost_test = np.sum((y_test - y_test_pred)**2)
print("Third order test cost:", cost_test)
third_order_test = cost_test
```

```
# fourth order model
P = np.array([x_train**0,x_train**1,x_train**2,x_train**3,x_train**4]).T
P_inv = np.linalg.pinv(P)
# Least squares solution
theta = P_inv @ y_train

# Calculate least squares cost on training data
y_train_pred = P @ theta
cost_train = np.sum((y_train - y_train_pred)**2)
print("Fourth order training cost:", cost_train)
fourth_order_train = cost_train

# Calculate least squares cost on test data
P_test = np.array([x_test**0, x_test**1, x_test**2, x_test**3,
x_test**4]).T
y_test_pred = P_test @ theta
cost_test = np.sum((y_test - y_test_pred)**2)
print("Fourth order test cost:", cost_test)
fourth_order_test = cost_test


# Create table of errors
def to_sci(x):
   return f"{x:.3e}"


# Save table data and model parameters to CSV
data = {
   'Model Order': ['First', 'Second', 'Third', 'Fourth'],
   'Theta': [
       ', '.join([to_sci(v) for v in np.linalg.pinv(np.array([x_train**0,
x_train**1]).T) @ y_train]),
       ', '.join([to_sci(v) for v in np.linalg.pinv(np.array([x_train**0,
x_train**1, x_train**2]).T) @ y_train]),
       ', '.join([to_sci(v) for v in np.linalg.pinv(np.array([x_train**0,
x_train**1, x_train**2, x_train**3]).T) @ y_train]),
       ', '.join([to_sci(v) for v in np.linalg.pinv(np.array([x_train**0,
x_train**1, x_train**2, x_train**3, x_train**4]).T) @ y_train])
   ],
```

```
    'Training Error': [to_sci(first_order_train),
to_sci(second_order_train), to_sci(third_order_train),
to_sci(fourth_order_train)],
    'Test Error': [to_sci(first_order_test), to_sci(second_order_test),
to_sci(third_order_test), to_sci(fourth_order_test)]
}
df = pd.DataFrame(data)
df.to_csv('errorTable.csv', index=False)

# Display and save table as image (without Theta column)
df_display = df.drop(columns=['Theta'])
fig, ax = plt.subplots(figsize=(6, 1.5))
ax.axis('off')
tbl = ax.table(cellText=df_display.values, colLabels=df_display.columns,
loc='center', cellLoc='center', colWidths=[0.35]*len(df_display.columns))
tbl.auto_set_font_size(False)
tbl.set_fontsize(12)
tbl.scale(1.0, 1.2)
plt.savefig('errorTable.png', bbox_inches='tight')
plt.show()

# Plot fits for each model order with actual data
plt.figure(figsize=(4, 2.5))
plt.scatter(x, y, color='black', label='Actual Data')

# First order fit
P_full_1 = np.array([x**0, x**1]).T
theta_1 = np.linalg.pinv(np.array([x_train**0, x_train**1]).T) @ y_train
y_fit_1 = P_full_1 @ theta_1
plt.plot(x, y_fit_1, label='First Order Fit', linestyle='--')

# Second order fit
P_full_2 = np.array([x**0, x**1, x**2]).T
theta_2 = np.linalg.pinv(np.array([x_train**0, x_train**1, x_train**2]).T)
@ y_train
y_fit_2 = P_full_2 @ theta_2
plt.plot(x, y_fit_2, label='Second Order Fit', linestyle='-.')

# Third order fit
P_full_3 = np.array([x**0, x**1, x**2, x**3]).T
```

```
theta_3 = np.linalg.pinv(np.array([x_train**0, x_train**1, x_train**2,
x_train**3]).T) @ y_train
y_fit_3 = P_full_3 @ theta_3
plt.plot(x, y_fit_3, label='Third Order Fit', linestyle=':')

# Fourth order fit
P_full_4 = np.array([x**0, x**1, x**2, x**3, x**4]).T
theta_4 = np.linalg.pinv(np.array([x_train**0, x_train**1, x_train**2,
x_train**3, x_train**4]).T) @ y_train
y_fit_4 = P_full_4 @ theta_4
plt.plot(x, y_fit_4, label='Fourth Order Fit', linestyle=(0, (3, 1, 1,
1)))

plt.xlabel('x')
plt.ylabel('y')
plt.legend()
plt.grid(True)
plt.savefig('modelPlots.png', bbox_inches='tight')
plt.show()
```